NORTHSTAR TECHNOLOGIES PVT. LTD.

Frontend Development Guidelines

Standards for building accessible, responsive, secure, and maintainable user interfaces

Document ID	ENG-GDE-008
Document Type	GUIDE
Access Scope	TEAM_ONLY
Department	Frontend Engineering
Version	1.0
Effective Date	04 November 2026
Owner	Frontend Engineering Team

INTERNAL TEAM GUIDANCE

This fictional demonstration document is prepared for the OnBoardIQ platform. It is intended for engineering reference, document-ingestion, and retrieval testing; it is not a formally approved corporate standard.

1. Purpose and Scope

These guidelines define the baseline practices for designing, implementing, testing, and maintaining frontend applications at Northstar Technologies Pvt. Ltd. They apply to web interfaces, browser-based applications, shared UI components, and frontend integrations maintained by the Frontend Engineering team.

- **Consistency:** create predictable user experiences through shared design patterns, components, and conventions.
- **Accessibility:** make interfaces usable by people with different abilities and interaction preferences.
- **Performance:** deliver responsive experiences across supported devices and network conditions.
- **Security:** treat all browser input and client-side state as untrusted; enforce authorization on the server.
- **Maintainability:** use modular architecture, readable code, automated checks, and documented decisions.

Audience: frontend engineers, UI engineers, reviewers, technical leads, and authorized engineering administrators. Access to this guide is restricted through the document's TEAM_ONLY access scope.

2. Frontend Engineering Principles

- Build user-centred interfaces: prioritize clarity, predictable navigation, useful feedback, and task completion.
- Prefer simple solutions: avoid unnecessary dependencies, abstractions, global state, and premature optimization.
- Use progressive enhancement where practical so essential content and actions remain robust.
- Keep components focused: a component should have a clear responsibility and a well-defined interface.
- Make state explicit: distinguish server state, local UI state, URL state, and persisted preferences.
- Design for failure: account for loading, empty, error, offline, and permission-denied states.
- Use consistent patterns: follow the approved design system rather than inventing new styles for common interactions.

3. Application Architecture and Project Structure

Organize frontend code so that features can evolve independently while shared concerns remain reusable. The exact directory structure may differ by framework, but responsibilities should remain clear.

Area	Responsibility	Guideline
Routes / Pages	Route composition and page-level data coordination	Keep page components focused on assembling feature components.
Features	Business-specific UI, hooks, validation, and workflows	Group related code by feature or domain.
Shared Components	Reusable buttons, dialogs, tables, forms, and layout primitives	Keep APIs consistent and avoid business-specific assumptions.
Services / API Client	HTTP calls, request configuration, and response mapping	Centralize base URLs, headers, and error normalization.
State Management	Shared client state and server-state coordination	Choose the least complex tool that meets the requirement.
Styles / Design Tokens	Colors, spacing, typography, themes, and responsive rules	Prefer design tokens and shared styling conventions.

- Separate presentation from data-fetching and business workflow logic where doing so improves testability.
- Avoid circular dependencies and oversized shared utility modules.
- Keep environment-specific configuration outside committed source code.
- Remove unused code, stale feature flags, and obsolete dependencies as part of maintenance.

4. Component Design and Reusability

- Use clear component names that describe their purpose and follow the project's naming convention.
- Define explicit props and types; avoid loosely typed inputs where stronger types are practical.
- Keep components small enough to understand, test, and review.
- Do not duplicate complex UI logic across pages; extract shared components or hooks when reuse is justified.
- Avoid creating abstractions for one-off cases unless they clarify a meaningful boundary.
- Use composition for flexible components instead of adding many unrelated boolean props.
- Keep side effects out of render logic and follow the framework's lifecycle and hook rules.
- Document non-obvious component behaviour, accessibility requirements, and important prop constraints.

A shared component should have a stable API, sensible defaults, keyboard support where relevant, and clear behaviour in loading, disabled, and error states.

5. HTML, CSS, and Design System Standards

- Use semantic HTML elements according to their meaning; do not use generic containers when a semantic element is available.
- Prefer CSS variables or design tokens for shared colors, spacing, typography, borders, and elevation.
- Use the team's agreed styling approach consistently; avoid mixing multiple styling systems without an approved reason.
- Avoid brittle selectors, excessive specificity, and unnecessary global styles.
- Use responsive layouts that adapt to content and viewport size rather than targeting only a small set of device widths.
- Maintain readable line lengths, appropriate contrast, visible focus indicators, and consistent spacing.
- Do not communicate meaning through color alone; include text, icons with labels, or other cues.
- Use a consistent icon set and ensure decorative icons are hidden from assistive technology where appropriate.
- Support browser zoom and text resizing without clipping essential content.

6. Responsive Design and Cross-Browser Support

- Design mobile-first or content-first when appropriate, then enhance layouts for larger viewports.
- Test common viewport widths and orientations, including narrow mobile screens and large desktop displays.
- Prevent horizontal overflow except in components where horizontal scrolling is intentional and usable.
- Ensure navigation, menus, dialogs, tables, and forms remain usable on touch devices.
- Use flexible units and layout systems such as Flexbox and CSS Grid where appropriate.
- Define the supported browser matrix for each product and test critical workflows against it.
- Use feature detection or supported fallbacks for browser capabilities that are not universally available.

7. Accessibility and Inclusive Interaction

Accessibility is a quality requirement, not a final styling task. Teams should target WCAG 2.2 Level AA where applicable and follow the organization's adopted accessibility review process.

- Use semantic landmarks, headings in a logical order, and descriptive page titles.
- Ensure all interactive controls are operable by keyboard and have a visible focus state.
- Associate form labels and instructions with their controls; announce validation errors clearly.
- Provide meaningful alternative text for informative images and empty alternative text for purely decorative images.
- Use accessible names for icon-only buttons and controls.

- Manage focus when opening and closing dialogs, menus, and other overlays.
- Do not rely solely on hover to reveal essential information or actions.
- Respect reduced-motion preferences and avoid unnecessary animation that can cause discomfort.
- Test important flows with keyboard navigation and, where feasible, screen readers.

8. Frontend Security

The browser is an untrusted environment. Client-side checks improve usability but must never replace backend authentication and authorization.

- Never embed private API keys, database credentials, signing secrets, or privileged service tokens in frontend bundles.
- Do not trust role, permission, price, ownership, or other security-sensitive values supplied by the client.
- Enforce access control and data validation on the backend for every protected operation.
- Avoid unsafe HTML injection. Sanitize untrusted content using an approved library and safe rendering patterns when HTML is unavoidable.
- Protect authentication flows using approved secure cookie, token, and session practices; do not store sensitive credentials in insecure browser storage.
- Use HTTPS in production and follow approved Content Security Policy and security-header guidance.
- Validate and constrain redirects, uploaded file types, file sizes, and user-controlled URLs.
- Avoid exposing sensitive information in URLs, browser logs, analytics events, or client-side error messages.
- Keep dependencies current and review package advisories before adopting or upgrading libraries.
- Do not disable browser security controls or certificate validation to work around development issues.

9. API Integration and Data Fetching

- Centralize API communication in a shared client or service layer rather than scattering raw network calls throughout components.
- Define typed request and response contracts where supported.
- Handle loading, success, empty, validation-error, unauthorized, forbidden, not-found, and server-error states.
- Set appropriate request timeouts and cancellation behaviour for requests that are no longer needed.
- Prevent duplicate submissions for actions that should occur once, and provide clear progress feedback.
- Use pagination or incremental loading for large collections.
- Do not assume network requests complete in the order they were started; guard against stale responses when necessary.
- Avoid logging full payloads that may contain personal or confidential data.
- Treat data returned by APIs as untrusted input and render it safely.

10. State Management and Forms

Choose state ownership deliberately. Keep state as close as practical to the components that need it, and introduce shared state only when multiple parts of the application genuinely depend on it.

- Use URL parameters for navigable state such as filters, search terms, and pagination when appropriate.
- Separate server state from transient interface state and use a suitable data-fetching/cache strategy.
- Avoid duplicating the same source of truth across component state, global stores, and browser storage.
- Validate forms on both client and server; client validation is for usability, not security.
- Provide labels, required-field indicators, helpful constraints, and field-specific error messages.
- Preserve user input after recoverable errors where appropriate.

- Prevent accidental data loss when navigating away from important unsaved forms, using a clear confirmation flow.

11. Performance and User Experience

- Measure real user experience where possible; optimize based on profiling and performance metrics.
- Optimize images with appropriate dimensions, formats, compression, and lazy loading where suitable.
- Reduce unnecessary JavaScript, avoid oversized dependencies, and split code by route or feature when beneficial.
- Avoid expensive rendering work and unnecessary component updates; memoize only when measurement or clear reasoning supports it.
- Use caching carefully and ensure stale data is refreshed according to product requirements.
- Set performance budgets for critical routes where the product requires them.
- Use skeletons or progress indicators for meaningful waits and avoid layout shifts where possible.
- Prioritize Core Web Vitals and perceived responsiveness for user-facing web applications.

12. Testing and Quality Assurance

Test level	Purpose
Unit tests	Verify component logic, hooks, utilities, formatters, and validation rules in isolation.
Component tests	Verify rendered output, user interactions, accessibility semantics, and state transitions.
Integration tests	Verify interactions between UI modules, API clients, routing, and state management.
End-to-end tests	Verify critical workflows in a browser, including authentication and key business journeys.
Visual regression tests	Detect unintended layout or styling changes in important shared components and pages.
Accessibility tests	Check semantic structure, keyboard behaviour, labels, contrast, and automated accessibility rules.

- Test successful and unsuccessful paths, boundary values, empty states, and permission-denied scenarios.
- Use stable selectors based on accessible roles and labels rather than fragile CSS implementation details.
- Keep tests deterministic and avoid dependence on production data or uncontrolled external services.
- Investigate flaky tests and fix their root cause.
- Run formatting, linting, type checks, and relevant tests before opening a pull request.

13. Code Review and Git Workflow

- Use the team's agreed branch naming, commit message, and pull request conventions.
- Keep changes focused and explain the user or engineering problem being solved.
- Include screenshots or short recordings for meaningful visual changes when useful.
- Describe testing performed, accessibility considerations, API dependencies, and rollout risks.
- Review correctness, maintainability, responsive behaviour, accessibility, security, and performance.
- Avoid committing secrets, local environment files, build output, or unrelated formatting changes.
- Address review feedback constructively and record significant design decisions.

14. Build, Deployment, and Release

- Use CI to run linting, type checks, tests, dependency checks, and production builds.
- Build reproducible artifacts and keep environment-specific configuration separate from source code.
- Verify API base URLs, feature flags, asset paths, and runtime configuration for each environment.

- Use approved deployment processes and ensure frontend and backend changes remain compatible during rollout.
- Validate critical journeys after deployment and monitor client-side errors and performance.
- Maintain a rollback or recovery plan for releases that introduce regressions.
- Do not publish source maps or debug information publicly unless explicitly approved and safely configured.

15. Analytics, Privacy, and Client-Side Data

- Collect only analytics data required for an approved product or operational purpose.
- Do not send passwords, authentication tokens, sensitive personal information, or confidential business content to analytics tools.
- Follow applicable consent, retention, and privacy requirements for cookies and tracking technologies.
- Mask or omit sensitive values from client-side telemetry and error reports.
- Use approved storage mechanisms and define retention behaviour for locally persisted preferences or cached data.
- Provide clear user feedback when data is saved, updated, removed, or unavailable.

16. Documentation and Ownership

- Maintain setup instructions, supported runtime versions, build commands, test commands, and environment variable names in the project README.
- Document shared components, design tokens, routing conventions, and important state-management decisions.
- Record significant architecture decisions with context, alternatives, and trade-offs.
- Keep API contracts and frontend integration notes aligned with backend changes.
- Assign clear ownership for shared UI libraries, design-system components, and critical frontend applications.

17. Frontend Change Checklist

Check	Expected evidence
Requirements	Acceptance criteria, responsive expectations, and edge cases are understood.
Architecture	Feature boundaries, component APIs, and state ownership are appropriate.
Accessibility	Semantic markup, keyboard operation, focus, labels, and contrast are reviewed.
Security	No secrets in bundles; backend authorization is relied upon; untrusted content is handled safely.
API behaviour	Loading, empty, success, error, and unauthorized states are handled.
Quality	Linting, type checks, relevant tests, and production build pass.
Performance	Images, bundles, rendering, and network usage are reasonable.
Release	Configuration, compatibility, deployment verification, and rollback are considered.

18. Roles, Exceptions, and Maintenance

Role	Responsibility
Frontend Engineer	Implement, test, document, and support frontend changes in accordance with this guide.
Reviewer / Senior Engineer	Review architecture, accessibility, security, performance, and maintainability.
Frontend Team Lead	Maintain frontend technical direction, shared standards, and component ownership.
Engineering Administrator	Support approved engineering governance and access administration.

A deviation from these guidelines should have a documented reason, an identified owner, an assessment of user, security, and operational risk, and an agreed review date. Exceptions affecting authentication, sensitive data, accessibility commitments, or production reliability require appropriate review.

The Frontend Engineering Team should review this guide periodically and update it when frameworks, browser support, design standards, security requirements, or engineering practices materially change.

Document control: ENG-GDE-008 | Version 1.0 | Effective 04 November 2026 | Access Scope: TEAM_ONLY | Department: Frontend Engineering

Demo notice: Northstar Technologies Pvt. Ltd. is a fictional company used for demonstration and OnBoardIQ document-ingestion testing. This guide is illustrative and does not constitute legal, regulatory, or formally approved corporate policy.